

ESP32 Display Firmware — Comprehensive Architecture Analysis

⚠ LEGACY REFERENCE — describes the **OLD ESP32-S3 board**. This document analyzes the legacy **Sunton ESP32-8048S043C (ESP32-S3)** firmware (GPIO 17/18 @ 2,000,000 baud). Current production has migrated to the **ESP32-P4** board (GPIO 27/28 @ 576,000 baud) — see the main [README](#) (`../README.md`) and [CHANGELOG_P4_UI_STABLE.md](#) (`CHANGELOG_P4_UI_STABLE.md`). Kept for historical reference on the old board's internals.

Project: Spot Welder Full

Branch: Dev

Firmware Target: ESP32-S3 (ESP32-8048S043C — 800×480 TFT + GT911 capacitive touch)

Analysis Date: 2 June 2026

Document Purpose: Encyclopedic reference for all ESP32 display firmware internals — project structure, UI architecture, communication protocol, settings management, code patterns, and integration points.

Table of Contents

1. [Project Structure & Build System](#)
 2. [Hardware Platform](#)
 3. [System Architecture Overview](#)
 4. [Boot Sequence & Initialization](#)
 5. [Main Loop Architecture](#)
 6. [UART Communication Protocol \(ESP32 ↔ STM32\)](#)
 7. [TCP/WiFi Bridge \(ESP32 ↔ Flask Server\)](#)
 8. [UI Architecture \(LVGL\)](#)
 9. [Lead Resistance Input — Complete Flow](#)
 10. [Settings Management & NVS Persistence](#)
 11. [Anti-Shudder System \(Touch Stability\)](#)
 12. [Draft/Apply Model \(Pulse Tab\)](#)
 13. [Callback Architecture](#)
 14. [Key Data Structures](#)
 15. [Code Patterns & Conventions](#)
 16. [Integration Points for Future Development](#)
 17. [INA226 Library \(Legacy\)](#)
 18. [Known Constraints & Caveats](#)
-

1. Project Structure & Build System

File Layout

```
Spotwelder Full/
├─ platformio.ini          (57 lines)  – Build configuration
├─ src/
│   └─ main.cpp            (2236 lines) – Entry point, UART parser, WiFi/
TCP bridge, NVS
│   └─ ui.cpp              (2289 lines) – LVGL UI construction, screens,
widgets, updates
├─ include/
│   └─ ui.h                (155 lines)  – UI public API: structs, call-
backs, declarations
│   └─ lv_conf.h           (384 lines)  – LVGL v9.1.0 configuration
├─ lib/
│   └─ INA226/
│       └─ INA226.h        (109 lines)  – Legacy INA226 I2C driver (likely
unused)
```

Total firmware: ~5,130 lines across 5 source files.

Build System (PlatformIO)

platformio.ini (lines 1-57):

```
[env:esp32-8048S043C]
platform = espressif32@^6.9.0
board = esp32-s3-devkitc-1
framework = arduino
board_build.arduino.memory_type = qio_opi
board_build.partitions = default_16MB.csv

lib_deps =
    rzldent/esp32_smartdisplay@^2.0.0

build_flags =
    -DBOARD_HAS_PSRAM
    -DARDUINO_USB_CDC_ON_BOOT=1
    -DARDUINO_USB_MODE=1
    -DDISPLAY_WIDTH=800
    -DDISPLAY_HEIGHT=480
    -DLV_CONF_INCLUDE_SIMPLE
    -DLV_FONT_MONTSERRAT_16=1
    -DLV_FONT_MONTSERRAT_18=1
    -DLV_FONT_MONTSERRAT_20=1
    -DLV_FONT_MONTSERRAT_24=1
    -DTOUCH_GT911_INT_ACTIVE_LEVEL=0
    -DTOUCH_GT911_INT_PRESS_INT_EN=0
```

Key Build Details

Setting	Value	Purpose
Platform	espressif32@^6.9.0	ESP-IDF + Arduino framework
Board	esp32-s3-devkitc-1	Generic ESP32-S3 (actual: ES-P32-8048S043C)
Memory	QIO OPI + PSRAM	Quad SPI flash, OctalSPI PSRAM
Partitions	default_16MB.csv	16 MB flash layout
Display lib	esp32_smartdisplay@^2.0.0	Handles TFT + touch driver init
LVGL version	v9.1.0	Confirmed in lv_conf.h header
Touch fix	GT911_INT_ACTIVE_LEVEL=0 , INT_PRESS_INT_EN=0	Polling mode (no interrupt)

Font Build Flags

Montserrat sizes 14, 16, 18, 20, 24, 48 are enabled. The lv_conf.h enables 14 as the default, with 16/18/20/24 used by UI widgets and 48 available for large displays.

2. Hardware Platform

ESP32-8048S043C Display Module

Feature	Specification
MCU	ESP32-S3 (dual-core Xtensa LX7, 240 MHz)
Display	4.3" TFT, 800×480 pixels, RGB565 color
Touch	GT911 capacitive touch controller
Flash	16 MB
PSRAM	OctalSPI (OPI mode)
USB	CDC on boot (serial over USB)

UART2 (STM32 Link)

Parameter	Value
Baud rate	2,000,000 (2 Mbps)
RX pin	GPIO 17
TX pin	GPIO 18
Buffer size	4096 bytes

Configured in `main.cpp` line ~2030:

```
STM32Serial.begin(2000000, SERIAL_8N1, 17, 18);
STM32Serial.setRxBufferSize(4096);
```

Physical Button

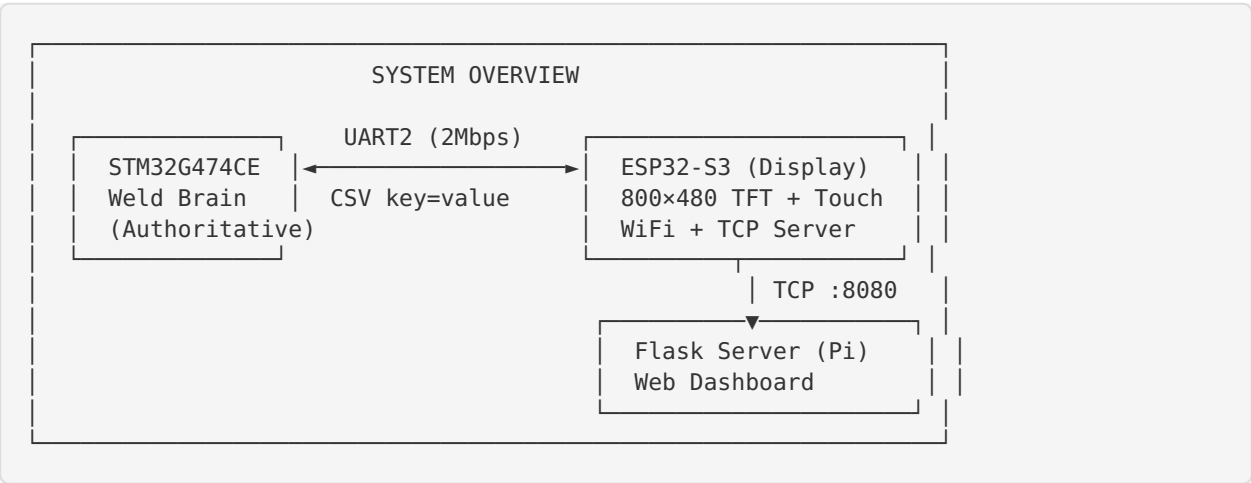
A GPIO boot button (active LOW, internal pullup) provides 3 gestures:

- **Single click** → Arm/Disarm toggle
- **Double click** → ESP32 restart
- **Long press (≥2s)** → Deep sleep

Defined at `main.cpp` line ~43:

```
#define BOOT_BUTTON_PIN 0
```

3. System Architecture Overview



The cardinal rule: The STM32 is **authoritative** for all weld-control state. The ESP32 is a display + bridge — it proposes settings, but the STM32 must ACK/DENY before the ESP32 persists them locally.

4. Boot Sequence & Initialization

The boot sequence is carefully orchestrated with a 4-phase structure and safety-critical defaults.

setup() — main.cpp lines 2020-2125

Phase 1: Hardware Init

- | | |
|-------------------------------|---|
| 1. Serial.begin(115200) | – USB debug console |
| 2. STM32Serial.begin(2000000) | – UART2 to STM32 |
| 3. smartdisplay_init() | – TFT + GT911 touch driver |
| 4. Touch indev wrapper | – Custom debounce filter wrapping GT911 |
| 5. ui_init(callbacks) | – Build all LVGL screens |

Phase 2: Config Load (NVS → Memory, NO STM32 sends)

```
// main.cpp lines 2053-2082
ConfigState cfg = load_config_from_nvs();
apply_brightness(cfg.brightness);    // Local display only
if (cfg.load_last_on_boot) {
    load_recipe_from_nvs();           // Into globals, NOT sent to STM32
}
stm_armed = false;                   // Safety: always boot DISARMED
trigger_mode = TRIGGER_MODE_PEDAL;   // Safety: always boot in PEDAL mode
contact_with_pedal = true;           // Safety: pedal gating enabled
load_lead_resistance_from_nvs();
ui_set_config_cb(onConfigChange);
ui_load_config(cfg);                 // Push config to UI widgets
```

Phase 3: Safety Defaults

Hardcoded at boot regardless of NVS values (main.cpp lines 2084-2102):

- stm_armed = false — DISARMED
- trigger_mode = PEDAL — Safest trigger mode
- contact_with_pedal = true — Pedal gating enabled

Phase 4: WiFi + TCP

```
// main.cpp lines 2112-2124
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
setup_done_ms = millis(); // Start fallback timer
```

STM32 Boot Handshake

After setup() , the system waits in loop() for the STM32:

1. **STM32 sends B00T, message** → ESP32 calls sendBootConfig() immediately
2. **Fallback:** If no B00T message within 3 seconds (B00T_TIMEOUT_MS), ESP32 sends config anyway

sendBootConfig() — 8-Step Paced Sequence

main.cpp lines ~1765-1830. Sends commands with 20ms gaps for STM32 processing:

Step 1: SET_PULSE,mode,d1,gap1,d2,gap2,d3	(weld recipe)
Step 2: SET_POWER,pct	(power percentage)
Step 3: SET_PREHEAT,en,ms,pct,gap	(preheat settings)
Step 4: SET_TRIGGER_MODE,1	(PEDAL – forced safe)
Step 5: ARM,0	(DISARMED – forced safe)
Step 6: SET_CONTACT_HOLD,steps	(contact hold time)
Step 7: SET_CONTACT_WITH_PEDAL,1	(pedal gating – forced safe)
Step 8: LEAD_R,ohms	(lead resistance)

After sending, a **2-second boot grace period** suppresses STATUS recipe-sync to prevent STM32 defaults from overwriting the ESP32's NVS-loaded values.

5. Main Loop Architecture

loop() — main.cpp lines 2131-2235

The main loop runs at ~200 Hz (5ms `delay()` at the end) and executes these tasks every iteration:

```

1. LVGL tick + timer handler
   lv_tick_inc() + lv_timer_handler()

2. Boot fallback check
   If no BOOT msg after 3s → sendBootConfig()

3. WiFi + TCP server maintenance
   ensureWiFiAndServer()

4. UART polling
   pollStm32Uart()

5. Ready heartbeat
   serviceReadyHeartbeat()

6. Dual-path voltage forwarding (TCP)
   STATUS2 at 500ms (raw, for graphs)
   DISPLAY at 1000ms (smoothed, for UI labels)

7. Periodic STATUS request (every 1s)
   requestStm32Status()

8. TCP client management
   Accept new, drop old, idle timeout, read cmds

9. Button handling
   handleButton() – single/double/long press

10. Touch state → UI
    ui_set_touch_active(hw_touch)

11. UI update
    updateScreenDisplay() → ui_update()

12. delay(5)

```

Voltage Display Smoothing

`main.cpp` lines ~20-60 define `VoltageDisplaySmoother` :

- 5mV quantized hysteresis (`snap` parameter)
- Prevents label flicker from ADC noise
- Used for `pack_voltage` display values sent to TCP clients

6. UART Communication Protocol (ESP32 ↔ STM32)

All messages are **newline-terminated CSV** strings with `key=value` pairs.

6.1 ESP32 → STM32 Commands

Command	Format	Description
<code>SET_PULSE</code>	<code>SET_PULSE,mode,d1,gap1,d2,gap2,d3</code>	Set weld pulse recipe
<code>SET_POWER</code>	<code>SET_POWER,pct</code>	Set power percentage (50-100)
<code>SET_PREHEAT</code>	<code>SET_PREHEAT,en,ms,pct,gap</code>	Preheat config
<code>SET_TRIGGER_MODE</code>	<code>SET_TRIGGER_MODE,mode</code>	1=Pedal, 2=Probe Contact
<code>SET_CONTACT_HOLD</code>	<code>SET_CONTACT_HOLD,steps</code>	Contact hold steps (1-10)
<code>SET_CONTACT_WITH_PEDAL</code>	<code>SET_CONTACT_WITH_PEDAL,val</code>	0/1 pedal gating
<code>LEAD_R</code>	<code>LEAD_R,%.6f</code>	Lead resistance in Ω (float)
<code>ARM</code>	<code>ARM,0</code> or <code>ARM,1</code>	Arm/disarm welder
<code>READY</code>	<code>READY,1</code>	ESP32 ready heartbeat
<code>STATUS</code>	<code>STATUS</code>	Request STATUS report
<code>FIRE</code>	<code>FIRE</code>	Trigger weld (passthrough from TCP)
<code>RESET_WELD_COUNT</code>	<code>RESET_WELD_COUNT</code>	Reset weld counter

6.2 STM32 → ESP32 Messages

Message	Format	Description
BOOT	BOOT,key=val,...	STM32 boot announcement
STATUS	STATUS,key=val,...	Periodic status with recipe echo
STATUS2	STATUS2,ina_ok=,chg_en=,vpack=,...	High-res voltage/current telemetry
ACK,*	ACK,SET_PULSE etc.	Command acknowledged
DENY,*	DENY,LEAD_R etc.	Command rejected
EVENT	EVENT,WELD_START / EVENT,WELD_DONE	Weld cycle events
WAVEFORM*	Raw waveform data	Passthrough to TCP client

6.3 STATUS Message Parsing

`pollStm32Uart()` at `main.cpp` lines ~800–1100 parses STATUS responses:

```
STATUS,mode=1,d1=5,gap1=0,d2=0,gap2=0,d3=0,power=100,armed=0,
welding=0,preheat_en=0,preheat_ms=20,preheat_pct=30,
preheat_gap=3,trigger_mode=1,contact_hold=2,weld_count=0
```

Recipe sync logic: Unless the boot grace period is active, STATUS recipe values overwrite local globals. This means the STM32's echoed settings become the source of truth.

6.4 STATUS2 Message Parsing

```
STATUS2,ina_ok=1,chg_en=1,vpack=9.230,ichg=0.45,cell1=3.080,
cell2=3.075,cell3=3.075,temp=25.3,weld_v=0.000,cap_v=9.230,
weld_v_b=0.000,weld_v_a=0.000,cap_v_b=0.000,cap_v_a=0.000,
energy_cap=0.00,energy_weld=0.00,energy_loss=0.00
```

6.5 UART Line Parser Implementation

`main.cpp` uses `extractFieldValue()`, `extractIntField()`, and `extractFloatField()` helpers (lines ~260–300) for key=value parsing:

```
String extractFieldValue(const String& line, const char* key) {
    String prefix = String(key) + "=";
    int idx = line.indexOf(prefix);
    if (idx < 0) return "";
    int start = idx + prefix.length();
    int end = line.indexOf(',', start);
    return (end < 0) ? line.substring(start) : line.substring(start, end);
}
```

7. TCP/WiFi Bridge (ESP32 ↔ Flask Server)

Connection Parameters

```
// main.cpp lines ~10-20
const char* ssid = "SpotWelder_AP";           // or configured WiFi
const char* password = "weld1234";
WiFiServer server(8080);                       // TCP port 8080
#define TCP_IDLE_TIMEOUT_MS 30000              // 30s idle disconnect
```

Dual-Path Voltage Forwarding

The TCP bridge sends two types of voltage data to the Flask server:

1. **Raw STATUS2** at 500ms intervals — High-resolution for graph/calculator consumers
2. **Smoothed DISPLAY** at 1000ms intervals — 5mV-quantized for stable UI labels

TCP Command Processing

Incoming TCP commands are processed through `processCommand()` (`main.cpp` lines ~1150-1450), which handles:

- `SET_PULSE` , `SET_POWER` , `SET_PREHEAT` → Forward to STM32
- `SET_LEAD_R` → STM32-authoritative flow (send, await ACK/DENY)
- `ARM` , `STATUS` , `FIRE` → Forward/handle
- Unknown commands → Passthrough to STM32

Client Management (`loop()` lines 2175-2218)

- Single TCP client at a time (new client drops old)
 - `setNoDelay(true)` for low-latency
 - On connect: sends current status + syncs settings
 - Idle timeout: drops client after 30s of no data
-

8. UI Architecture (LVGL)

8.1 Display Framework

Parameter	Value
Library	LVGL v9.1.0
Color depth	RGB565 (16-bit)
Memory pool	128 KB (increased from default 64 KB)
Refresh rate	~60 fps (16ms period)
DPI	130
Theme	Dark (custom #1A1A2E background)
OS integration	None (LV_OS_NONE)

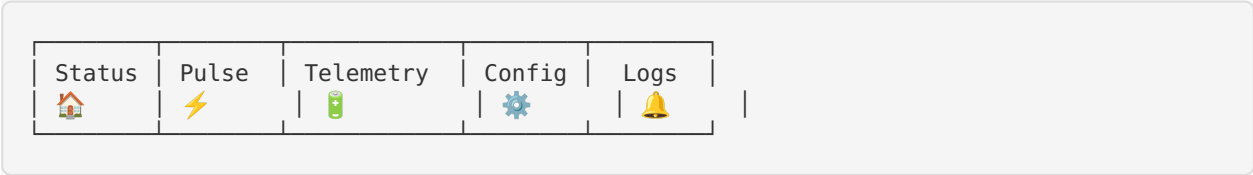
8.2 Color Scheme

Defined in ui.cpp lines 59–68:

```
#define C_BG          lv_color_hex(0x1A1A2E) // Dark navy background
#define C_CARD        lv_color_hex(0x222240) // Card/panel background
#define C_ACCENT       lv_color_hex(0xFF6600) // Orange accent
#define C_GREEN        lv_color_hex(0x00CC66) // Positive/armed/on
#define C_RED          lv_color_hex(0xFF3333) // Negative/disarmed/danger
#define C_YELLOW       lv_color_hex(0xFFDD44) // Warning/pending
#define C_WHITE        lv_color_hex(0xFFFFFF) // Primary text
#define C_GREY         lv_color_hex(0x888888) // Secondary text/labels
#define C_DARK_GREY    lv_color_hex(0x333355) // Inactive buttons/borders
```

8.3 Tab Structure

ui_init() creates a 5-tab tabview (ui.cpp lines 1828–1899):



- **Tab bar:** 42px tall, top position, lv_font_montserrat_14
- **Swipe disabled:** Tab content area has scrolling/gesture cleared to prevent shudder
- **Size:** 800×480 full screen

8.4 Tab 0: Status Tab

Built by build_status_tab() (ui.cpp lines 573–737).

Layout:

[Weld Count ]		200px×52px	Trigger: [Pedal] [Probe]	
"0"			120px×44px each	
Pack Voltage 9.23 V		Temperature 25.3 °C	Charger Current 0.45 A	
Cell 1 3.080 V	Cell 2 3.075 V	Cell 3 3.075 V		
[DISARMED (tap to arm)] 260×56px Red when disarmed, Green when armed				

Key widgets:

- **Weld Counter** — Clickable card, tap to reset (fires `_weld_reset_cb`)
- **Trigger Buttons** — Two separate buttons: Pedal (orange when active) / Probe (green when active)
- **Value Cards** — `make_card()` helper creates titled cards with value labels
- **ARM Button** — Plain `lv_obj` (Widget A pattern), 260×56px, color-coded

8.5 Tab 1: Pulse Tab

Built by `build_pulse_tab()` (`ui.cpp` lines 1051–1289).

Two-column layout:

LEFT COLUMN		RIGHT COLUMN	
▶ Weld Mode [Single] [Double] [Triple]		▶ Weld Power ▶ Preheat [-] 100% [+] [ON/OFF]	
▶ Pulse Timing Pulse [-] [005] [+] ms Gap [-] [000] [+] ms Pulse 2 [-] [000] [+] ms ← hide Gap 2 [-] [000] [+] ms ← hide Pulse 3 [-] [000] [+] ms ← hide		Duration [-] [020] [+] ms Power [-] [030] [+] % Gap [-] [003] [+] ms	
		[Apply Settings] 200×50px ▲ Changes pending	

Key features:

- **Mode buttons** toggle which timing rows are visible (Single=d1 only, Double=+gap1+d2, Triple=all)
- **Spinbox rows** created by `make_touch_row()` — 56×46px +/- buttons, 84×46px value display
- **Power** — +/- buttons with config-driven step size (1%, 5%, or 10%), range 50–100%
- **Preheat** — Toggle ON/OFF, reveals/hides Duration/Power/Gap rows
- **Apply** — Sends recipe to STM32 via callback; green when synced, red when pending changes
- **Hold-to-repeat** — All +/- buttons support press-and-hold (400ms initial delay, 120ms repeat)
- **Lockable controls** — All interactive widgets registered in `lockable_objs[]` array

8.6 Tab 2: Telemetry Tab

Built by `build_telemetry_tab()` (`ui.cpp` lines 1294–1330).

2×3 grid of diagnostic cards:

Row	Left Card	Right Card
1	Weld Voltage	Weld V Drop
2	Cap V Drop	E Cap (J)
3	E Weld (J)	E Loss (J)

Each card is $(780-10)/2 = 385\text{px}$ wide $\times$ 106px tall. Weld V Drop turns red above 0.30V; Cap V Drop turns red above 0.40V.

8.7 Tab 3: Config Tab

Built by `build_config_tab()` (`ui.cpp` lines 1636–1823).

Scrollable container with 4 sections:

► Editing		
Hold-to-repeat (+/- buttons)	[ON/OFF]	(green/grey)
Time Step (timing +/-)	[1 ms]	(orange cycle)
Power Step (power +/-)	[5 %]	(orange cycle)
► Trigger		
Contact/Probe Hold Time	[1.0 s]	(cycle 0.5–5s)
Contact With Pedal	[ON/OFF]	(green/grey)
Lead Resistance (mΩ)	[ 2.0 mΩ]	slider 0.5–5.0, 0.1 step
► Startup		
Load last settings on boot	[ON/OFF]	(green/grey)
► Display		
Screen Brightness	[HIGH]	(cycle L/M/H)

Config controls use cycle-through buttons (tap to advance): Time Step cycles 1→5→10→1 ms, Power Step cycles 1→5→10→1 %, Brightness cycles LOW→MED→HIGH→LOW, Contact Hold cycles 0.5s→5.0s in 0.5s steps.

Lead Resistance uses an `lv_slider` with range 5–50 (representing 0.5–5.0 mΩ at 0.1 mΩ resolution). Updates on `VALUE_CHANGED` (visual) and `RELEASED` (notify config change).

8.8 Tab 4: Logs Tab

Placeholder only (`build_placeholder_tab()` , line 1335):

"Logs
(coming in Phase 2)"

8.9 Widget Construction Patterns

`make_card()` — Value Display Card

`ui.cpp` lines 541–568. Creates a rounded rectangle with title (grey, 14pt) and value label (green, configurable font).

```
static lv_obj_t* make_card(lv_obj_t* parent, const char* title,
                          lv_obj_t** out_value_label, int w, int h,
                          const lv_font_t* val_font = &lv_font_montserrat_14);
```

`make_touch_row()` — Spinbox with +/– Buttons

`ui.cpp` lines 862–973. Layout: [Label 90px] [– 56×46] [Value 84×46] [+ 56×46] [unit 40px]

- Spinbox is NOT directly clickable (cursor disabled)
- +/– buttons use hold-to-repeat with `HoldRepeatCtx` pool
- All widgets registered as lockable and interaction-safe

`make_cfg_button()` — Config Toggle Button

`ui.cpp` lines 1353–1376. Widget A pattern button with centered label.

`make_section_header()` — Section Title

`ui.cpp` lines 1009–1017. Orange text, 18pt font.

9. Lead Resistance Input — Complete Flow

The lead resistance setting has a unique **STM32-authoritative** flow that differs from other settings.

9.1 UI Layer (Config Tab)

Slider definition (`ui.cpp` lines 1754–1779):

```
static const float CFG_LEAD_R_MIN_MOHM = 0.5f; // 0.5 mΩ minimum
static const float CFG_LEAD_R_MAX_MOHM = 5.0f; // 5.0 mΩ maximum
static const int CFG_LEAD_R_SCALE = 10; // 0.1 mΩ resolution

// Slider range: 5–50 (integer), representing 0.5–5.0 mΩ
lv_slider_set_range(slider_cfg_lead_r,
  (int)lroundf(CFG_LEAD_R_MIN_MOHM * CFG_LEAD_R_SCALE), // 5
  (int)lroundf(CFG_LEAD_R_MAX_MOHM * CFG_LEAD_R_SCALE)); // 50
```

Slider event handler (`ui.cpp` lines 1613–1631):

```
static void on_cfg_lead_r_slider(lv_event_t* e) {
    lv_event_code_t code = lv_event_get_code(e);
    if (code != LV_EVENT_VALUE_CHANGED && code != LV_EVENT_RELEASED) return;

    int raw = lv_slider_get_value(slider_cfg_lead_r);
    float mohm = clamp_cfg_lead_r_mohm((float)raw / (float)CFG_LEAD_R_SCALE);

    if (fabsf(_cfg.lead_resistance_mohm - mohm) > 0.0001f) {
        _cfg.lead_resistance_mohm = mohm;
    }

    update_cfg_lead_r_label();

    if (code == LV_EVENT_RELEASED) {
        notify_config_changed(); // → main.cpp onConfigChange()
    }
}
```

- `VALUE_CHANGED` — Updates label in real-time during drag
- `RELEASED` — Triggers `notify_config_changed()` → `onConfigChange()` callback

9.2 Callback Layer (main.cpp)

`onConfigChange()` (`main.cpp` lines ~1840-1870):

```
void onConfigChange(const ConfigState& cfg) {
    // ... brightness, contact_hold_steps handling ...

    // Lead resistance: detect change → send to STM32
    if (fabsf(cfg.lead_resistance_mohm - lead_resistance_mohm_nvs) > 0.001f) {
        lead_resistance_mohm = cfg.lead_resistance_mohm;
        String cmd = "SET_LEAD_R," + String(lead_resistance_mohm, 1);
        processCommand(cmd);
    }

    save_config_to_nvs(cfg);
}
```

9.3 processCommand("SET_LEAD_R") — `main.cpp` lines ~1220-1260

```
// Extract mΩ value, clamp to protocol range [0.1, 10.0]
float new_mr = cmd.substring(10).toFloat();
new_mr = constrain(new_mr, 0.1f, 10.0f);

// Convert mΩ to Ω for UART
float ohms = new_mr / 1000.0f;

// Send to STM32 — DO NOT persist yet (wait for ACK)
sendLeadResistanceToStm32(ohms);
```

9.4 sendLeadResistanceToStm32() — main.cpp lines ~1680-1690

```
void sendLeadResistanceToStm32(float ohms) {
    char buf[32];
    snprintf(buf, sizeof(buf), "LEAD_R,%.6f", ohms);
    forwardToStm32(String(buf));
    Serial.printf("[LEAD_R] Sent to STM32: %s\n", buf);
}
```

9.5 STM32 Response Handling

ACK path (main.cpp line ~950):

```
if (line.startsWith("ACK,LEAD_R")) {
    Serial.println("[ACK] STM32 accepted LEAD_R");
    save_lead_resistance_to_nvs(lead_resistance_mohm); // NOW persist
}
```

DENY path (main.cpp line ~960):

```
if (line.startsWith("DENY,LEAD_R")) {
    Serial.println("[DENY] STM32 rejected LEAD_R – reverting");
    load_lead_resistance_from_nvs(); // Revert to last known-good
}
```

9.6 NVS Persistence

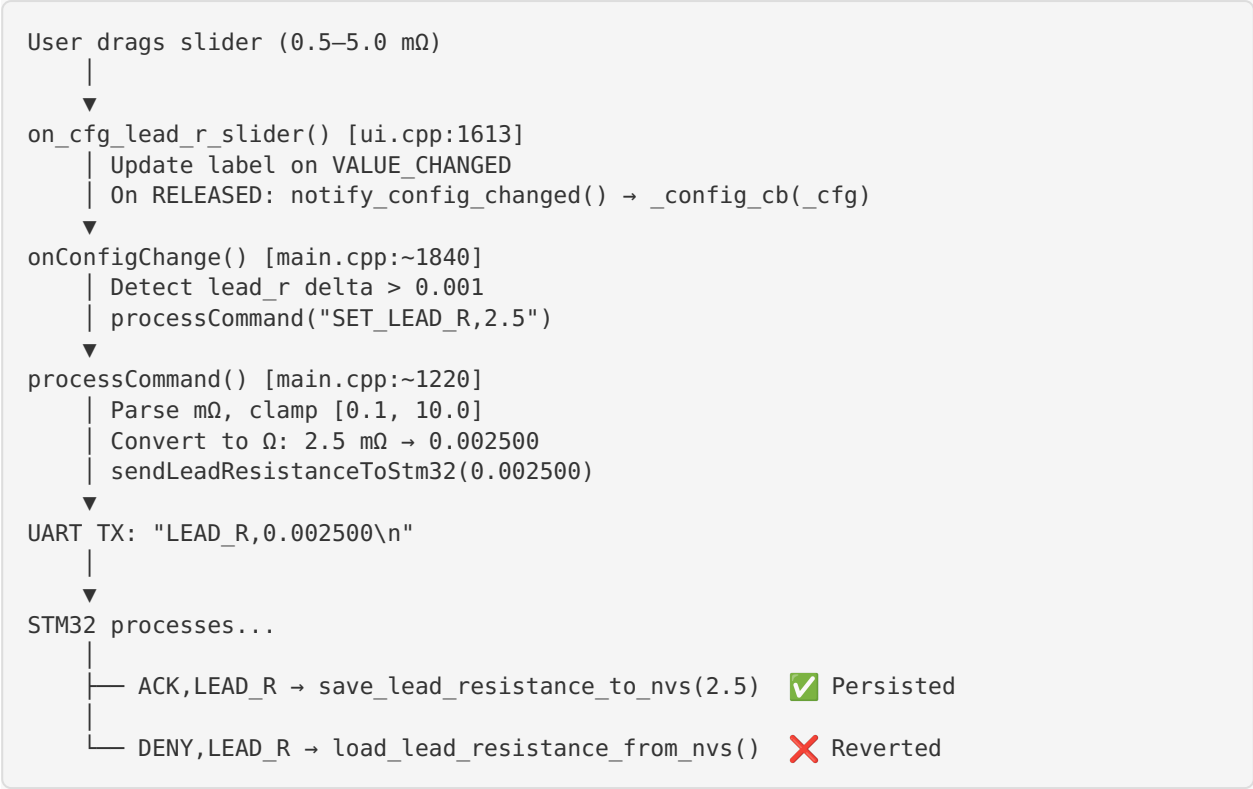
save_lead_resistance_to_nvs() (main.cpp lines ~1660-1675):

Stores in **two NVS namespaces** for backward compatibility:

```
void save_lead_resistance_to_nvs(float mohm) {
    // Primary: "welddcfg" namespace, key "lead_r_mohm"
    Preferences p;
    p.begin("welddcfg", false);
    p.putFloat("lead_r_mohm", mohm);
    p.end();

    // Legacy: "spotwelder" namespace, key "lead_r"
    p.begin("spotwelder", false);
    p.putFloat("lead_r", mohm);
    p.end();
}
```

9.7 Complete Data Flow Diagram



9.8 Unit Conversion Summary

Location	Unit	Range	Notes
UI slider	integer × 0.1	5-50 (0.5-5.0 mΩ)	CFG_LEAD_R_SCALE = 10
ConfigState	mΩ (float)	0.5-5.0	lead_resistance_mohm
NVS storage	mΩ (float)	0.5-5.0	Keys: lead_r_mohm / lead_r
UART protocol	Ω (float)	0.0001-0.01	LEAD_R,%.6f (divided by 1000)
processCommand	mΩ (float)	0.1-10.0	Protocol-level clamp range

10. Settings Management & NVS Persistence

10.1 NVS Namespaces

The ESP32 uses Arduino Preferences library with three NVS namespaces:

Namespace	Purpose	Keys
weldcfg	Config tab settings	hold_rep , time_step , pwr_step , load_last , bright , cwp , hold_steps , lead_r_mohm
weldrecipe	Pulse recipe	mode , d1 , gap1 , d2 , gap2 , d3 , power , ph_en , ph_ms , ph_pct , ph_gap
spotwelder	Legacy lead_r storage	lead_r

10.2 save_config_to_nvs() — main.cpp lines ~1550-1590

```
void save_config_to_nvs(const ConfigState& cfg) {
    Preferences prefs;
    prefs.begin("weldcfg", false);
    prefs.putBool("hold_rep", cfg.hold_to_repeat);
    prefs.putUChar("time_step", cfg.time_step_ms);
    prefs.putUChar("pwr_step", cfg.power_step_pct);
    prefs.putBool("load_last", cfg.load_last_on_boot);
    prefs.putUChar("bright", cfg.brightness);
    prefs.putBool("cwp", cfg.contact_with_pedal);
    prefs.putUChar("hold_steps", cfg.contact_hold_steps);
    prefs.putFloat("lead_r_mohm", cfg.lead_resistance_mohm);
    prefs.end();
}
```

10.3 load_config_from_nvs() — main.cpp lines ~1600-1640

Returns `ConfigState` with defaults for any missing keys. Uses `config_defaults()` from `ui.h` line 66.

10.4 Recipe Persistence

save_recipe_to_nvs() — Called after `onRecipeApply()` (not after every edit, only after Apply).

load_recipe_from_nvs() — Called during boot if `load_last_on_boot` is enabled. Loads into global variables (`weld_mode`, `weld_d1`, etc.) but does NOT send to STM32 yet.

10.5 Config Defaults

Defined in `ui.h` lines 66-77:

```
static inline ConfigState config_defaults() {
    ConfigState c;
    c.hold_to_repeat    = true;
    c.time_step_ms      = 1;      // 1 ms steps
    c.power_step_pct    = 5;      // 5% power steps
    c.load_last_on_boot = true;   // Restore recipe at startup
    c.brightness        = 2;      // HIGH
    c.contact_with_pedal = false;
    c.contact_hold_steps = 2;      // 1.0 seconds
    c.lead_resistance_mohm = 2.0f; // 2.0 mΩ
    return c;
}
```

11. Anti-Shudder System (Touch Stability)

The anti-shudder system is a critical piece of the UI architecture, solving LVGL display artifacts caused by concurrent touch processing and label updates.

11.1 Root Cause

`lv_button_create()` (themed LVGL widget) causes visible shudder every 2-3 presses due to LVGL's GROW/transition animations. Full-screen redraws compete with touch input processing.

11.2 Solution: “Widget A” Pattern

ALL interactive controls use plain `lv_obj_create()` instead of `lv_button_create()`:

```
lv_obj_t* btn = lv_obj_create(parent);
lv_obj_remove_style_all(btn);           // Strip ALL theme styles
lv_obj_add_flag(btn, LV_OBJ_FLAG_CLICKABLE); // Make it clickable
lv_obj_clear_flag(btn, LV_OBJ_FLAG_SCROLLABLE); // No scrolling
// Apply manual styling (bg color, radius, etc.)
```

11.3 `make_interaction_safe()` — `ui.cpp` lines 133-146

Applied to **every** interactive widget:

```
static void make_interaction_safe(lv_obj_t* obj) {
    // Stop event bubbling for ALL touch events
    lv_obj_add_event_cb(obj, on_stop_bubble, LV_EVENT_PRESSED, nullptr);
    lv_obj_add_event_cb(obj, on_stop_bubble, LV_EVENT_PRESSING, nullptr);
    lv_obj_add_event_cb(obj, on_stop_bubble, LV_EVENT_RELEASED, nullptr);
    lv_obj_add_event_cb(obj, on_stop_bubble, LV_EVENT_GESTURE, nullptr);
    // Track touch state
    lv_obj_add_event_cb(obj, on_touch_begin, LV_EVENT_PRESSED, nullptr);
    lv_obj_add_event_cb(obj, on_touch_end, LV_EVENT_RELEASED, nullptr);
    // Prevent gesture/scroll propagation to parent
    lv_obj_clear_flag(obj, LV_OBJ_FLAG_GESTURE_BUBBLE);
    lv_obj_clear_flag(obj, LV_OBJ_FLAG_SCROLL_CHAIN_HOR);
}
```

11.4 Touch-Gated UI Updates

`ui_update()` (`ui.cpp` lines 1946-1967) blocks all visual updates during touch:

```

bool any_touch = _hw_touch_active || _widget_touch_active;
if (any_touch) {
    return; // Completely skip – no redraws during touch
}

// Post-touch cooldown: reduced rate briefly after release
uint32_t interval = 100; // 10 Hz default
if (now - _touch_release_ms < 300) {
    interval = 250; // 4 Hz briefly after release (300ms cooldown)
}

```

11.5 Dual Touch Tracking

Two independent flags provide belt-and-suspenders coverage:

1. `_hw_touch_active` — Set by `ui_set_touch_active()` from `main.cpp`’s hardware touch driver state machine (catches ALL touches including non-widget areas)
2. `_widget_touch_active` — Set by `on_touch_begin` / `on_touch_end` LVGL event callbacks (catches widget-specific touches)

11.6 LVGL Theme Optimization

In `lv_conf.h` line 287:

```

#define LV_THEME_DEFAULT_GROW 0 // Disable button grow animation
#define LV_THEME_DEFAULT_TRANSITION_TIME 0 // Zero transition time

```

11.7 Tabview Anti-Shudder

`ui_init()` lines 1847–1859 disable all scroll/gesture on the tabview content:

```

lv_obj_clear_flag(tv_content, LV_OBJ_FLAG_SCROLLABLE);
lv_obj_clear_flag(tv_content, LV_OBJ_FLAG_SCROLL_ELASTIC);
lv_obj_clear_flag(tv_content, LV_OBJ_FLAG_GESTURE_BUBBLE);
lv_obj_set_scroll_dir(tv_content, LV_DIR_NONE);

```

12. Draft/Apply Model (Pulse Tab)

12.1 Concept

The Pulse tab uses a **draft/applied** state model:

- **Draft** values are what the user sees and edits
- **Applied** values reflect what the STM32 is actually using
- A visible “Changes pending” indicator appears when draft ≠ applied

12.2 State Variables

`ui.cpp` lines 341–366:

```
// Draft (user-editable)
static uint8_t draft_mode, draft_d1, draft_gap1, draft_d2, ...
static uint8_t draft_power;
static bool draft_preheat_en;
// ... etc

// Applied (from STM32 STATUS echo)
static uint8_t applied_mode, applied_d1, applied_gap1, ...

static bool draft_dirty = false;           // draft ≠ applied
static bool applied_initialized = false;
```

12.3 Apply Flow

1. User edits draft values (mode, timing, power, preheat)
2. `mark_dirty()` called → `draft_dirty = true` → Apply button turns red, “Changes pending” shown
3. User taps **Apply** → `on_apply_click()` → fires `_recipe_cb(draft values)` → `onRecipeApply()` in `main.cpp`
4. `onRecipeApply()` sends `SET_PULSE`, `SET_POWER`, `SET_PREHEAT` to STM32 + saves to NVS
5. STM32 echoes back via STATUS → `sync_applied_from_state()` updates applied values
6. `update_draft_dirty()` → `draft_dirty = false` → Apply button turns green

12.4 Auto-Sync on Boot

`sync_applied_from_state()` (`ui.cpp` lines 386–473) has special logic:

- **First time** (`!applied_initialized`): copies applied → draft unconditionally
- **Subsequently**: only syncs draft from applied if user hasn’t manually edited (`!draft_dirty`)
- This prevents false “unsaved changes” indicators after boot config echo

13. Callback Architecture

13.1 Callback Types (defined in `ui.h`)

Type	Signature	Purpose
<code>arm_toggle_cb_t</code>	<code>void(bool arm)</code>	Arm/disarm toggle from UI
<code>recipe_apply_cb_t</code>	<code>void(mode, d1, gap1, d2, gap2, d3, power, ph_en, ph_ms, ph_pct, ph_gap)</code>	Recipe apply from Pulse tab
<code>config_change_cb_t</code>	<code>void(const ConfigState& cfg)</code>	Config tab change
<code>trigger_source_cb_t</code>	<code>void(uint8_t trigger_mode)</code>	Trigger mode change
<code>weld_count_reset_cb_t</code>	<code>void(void)</code>	Weld counter reset tap
<code>contact_with_pedal_cb_t</code>	<code>void(bool enabled)</code>	Contact-with-pedal toggle

13.2 Registration (in `main.cpp setup()`)

```
// During ui_init:
ui_init(onArmToggle, onRecipeApply);           // lines ~2040

// After config load:
ui_set_config_cb(onConfigChange);              // line 2077

// After safety defaults:
ui_set_trigger_source_cb(onTriggerSourceChange); // line 2105
ui_set_weld_count_reset_cb(onWeldCountReset);    // line 2106
ui_set_contact_with_pedal_cb(onContactWithPedalChange); // line 2107
```

13.3 Callback Implementations (main.cpp)

Callback	Action
<code>onArmToggle(bool)</code>	Sends <code>ARM,0/1</code> to STM32
<code>onRecipeApply(...)</code>	Sends <code>SET_PULSE + SET_POWER + SET_PREHEAT</code> to STM32, saves NVS
<code>onConfigChange(cfg)</code>	Handles brightness, contact_hold, lead_r changes; saves config NVS
<code>onTriggerSourceChange(mode)</code>	Sends <code>SET_TRIGGER_MODE,mode</code> to STM32
<code>onWeldCountReset()</code>	Sends <code>RESET_WELD_COUNT</code> to STM32
<code>onContactWithPedalChange(en)</code>	Sends <code>SET_CONTACT_WITH_PEDAL,0/1</code> to STM32

14. Key Data Structures

14.1 `WelderDisplayState` (`ui.h` lines 12-49)

Passed from `main.cpp` → `ui_update()` every cycle:

```

struct WelderDisplayState {
    // Pack/charger telemetry
    float pack_voltage, temperature, charger_current;
    float cell1_v, cell2_v, cell3_v;

    // Weld voltage/energy telemetry (Phase 1B)
    float weld_v, cap_v;
    float weld_v_b, weld_v_a;           // Before/after voltages
    float cap_v_b, cap_v_a;
    float weld_v_drop, cap_v_drop;
    float energy_cap_j, energy_weld_j, energy_loss_j;

    // State flags
    bool armed, welding, charging;
    uint32_t weld_count;

    // Recipe (authoritative from main.cpp globals)
    uint8_t weld_mode;
    uint16_t pulse_d1, pulse_gap1, pulse_d2, pulse_gap2, pulse_d3;
    uint8_t power_pct;
    bool preheat_enabled;
    uint16_t preheat_ms, preheat_gap_ms;
    uint8_t preheat_pct;

    // Trigger settings
    uint8_t trigger_mode;
    uint8_t contact_hold_steps;
};

```

14.2 ConfigState (ui.h lines 54-63)

```

struct ConfigState {
    bool hold_to_repeat;           // Hold +/- to auto-repeat
    uint8_t time_step_ms;         // 1, 5, or 10 ms
    uint8_t power_step_pct;       // 1, 5, or 10 %
    bool load_last_on_boot;       // Restore recipe at startup
    uint8_t brightness;          // 0=LOW, 1=MED, 2=HIGH
    bool contact_with_pedal;      // Require pedal for contact trigger
    uint8_t contact_hold_steps;   // 1-10 (0.5s per step)
    float lead_resistance_mohm;   // 0.5-5.0 mΩ
};

```

14.3 HoldRepeatCtx (ui.cpp lines 155-161)

Context for hold-to-repeat buttons:

```

struct HoldRepeatCtx {
    lv_obj_t* spinbox;           // Target spinbox (nullptr for power buttons)
    bool is_increment;           // true = +, false = -
    bool is_power;               // true = power button (custom handler)
    uint32_t press_start;        // millis() when pressed
    uint32_t last_repeat;        // millis() of last repeat action
};

```

Pool of 20 contexts (MAX_REPEAT_BTNS) allocated statically.

14.4 Touch Filter State Machine (`main.cpp`)

4-state debounce filter for GT911 touch:

```
IDLE → PENDING_PRESS → PRESSED → PENDING_RELEASE → IDLE
```

States (`main.cpp` lines ~200–250):

- **IDLE** — No touch, no output
- **PENDING_PRESS** — Touch detected, waiting for debounce confirmation
- **PRESSED** — Confirmed touch, reporting to LVGL
- **PENDING_RELEASE** — Touch released, waiting for debounce before clearing

Coordinate scaling: GT911 raw (470×265) → display (800×480).

15. Code Patterns & Conventions

15.1 Widget A Pattern

All interactive UI elements follow this strict pattern to avoid LVGL theme-induced shudder:

```
lv_obj_t* btn = lv_obj_create(parent);
lv_obj_remove_style_all(btn);           // 1. Strip all theme styles
lv_obj_add_flag(btn, LV_OBJ_FLAG_CLICKABLE); // 2. Make clickable
lv_obj_clear_flag(btn, LV_OBJ_FLAG_SCROLLABLE); // 3. Disable scrolling
lv_obj_set_size(btn, w, h);             // 4. Set size
lv_obj_set_style_bg_color(btn, color, 0); // 5. Manual bg color
lv_obj_set_style_bg_opa(btn, LV_OPA_COVER, 0); // 6. Opaque background
lv_obj_set_style_radius(btn, r, 0);      // 7. Corner radius
make_interaction_safe(btn);              // 8. Anti-shudder events
```

15.2 Float Formatting

LVGL's built-in `lv_snprintf` does NOT support `%f` (`LV_SPRINTF_USE_FLOAT = 0`). All float formatting uses standard C `snprintf()` :

```
char buf[32];
snprintf(buf, sizeof(buf), "%.2f V", (double)st.pack_voltage);
lv_label_set_text(lbl_pack_v, buf);
```

15.3 Change-Detection Updates

`ui_update()` uses static “previous value” variables to avoid redundant label updates:

```
static float prev_pack_v = -999.0f;
if (first_run || fabsf(st.pack_voltage - prev_pack_v) >= 0.01f) {
    // Update label only when value actually changed
    prev_pack_v = st.pack_voltage;
}
```

15.4 UART CSV Protocol Parsing

```
String extractFieldValue(const String& line, const char* key) {
    String prefix = String(key) + "=";
    int idx = line.indexOf(prefix);
    if (idx < 0) return "";
    int start = idx + prefix.length();
    int end = line.indexOf(',', start);
    return (end < 0) ? line.substring(start) : line.substring(start, end);
}
```

15.5 Safety-First Boot

Every boot enforces three safety defaults regardless of NVS values:

1. `stm_armed = false` — DISARMED
2. `trigger_mode = PEDAL` — Manual pedal mode
3. `contact_with_pedal = true` — Pedal gating enabled

16. Integration Points for Future Development

16.1 Adding a New Config Setting

1. **Add field to** `ConfigState` in `ui.h`
2. **Set default** in `config_defaults()` in `ui.h`
3. **Add NVS key** in `save_config_to_nvs()` / `load_config_from_nvs()` in `main.cpp`
4. **Build UI widget** in `build_config_tab()` in `ui.cpp` (follow existing pattern)
5. **Add event handler** in `ui.cpp` (e.g., `on_cfg_new_setting()`)
6. **Handle in** `onConfigChange()` in `main.cpp` if STM32 interaction needed

16.2 Adding a New Tab

1. Add `lv_tabview_add_tab(tv, "Icon Name")` in `ui_init()` (`ui.cpp` line ~1871)
2. Create `build_new_tab()` function following existing patterns
3. Disable scrolling/gesture on the new tab
4. Call builder from `ui_init()`

16.3 Adding a Calibration Button (Example)

To add a “Calibrate Lead Resistance” button to the Config tab:

1. **UI widget** — In `build_config_tab()`, after the lead resistance slider:

```
cpp
lv_obj_t* btn_cal = make_cfg_button(cont, "Calibrate", &lbl_cal,
                                   BTN_X, y, BTN_W, BTN_H, C_ACCENT);
lv_obj_add_event_cb(btn_cal, on_calibrate_lead_r, LV_EVENT_CLICKED, nullptr);
```

2. **New callback type** in `ui.h`:

```
cpp
typedef void (*calibrate_cb_t)(void);
void ui_set_calibrate_cb(calibrate_cb_t cb);
```

3. **Event handler** in `ui.cpp`:

```
cpp
```



```
static calibrate_cb_t _cal_cb = nullptr;
static void on_calibrate_lead_r(lv_event_t* e) {
    if (lv_event_get_code(e) != LV_EVENT_CLICKED) return;
    if (_cal_cb) _cal_cb();
}
```

4. main.cpp handler:

```
cpp
void onCalibrate() {
    forwardToStm32("CALIBRATE_LEAD_R");
    // Wait for ACK,CALIBRATE with new lead_r value
}
ui_set_calibrate_cb(onCalibrate);
```

5. **STM32 protocol extension:** Add `CALIBRATE_LEAD_R` command and `ACK,CALIBRATE,lead_r_mohm=X.X` response.

16.4 Adding a New UART Command

1. **ESP32→STM32** — Add case in `processCommand()` (`main.cpp` line ~1150)
2. **STM32→ESP32** — Add parsing in `pollStm32Uart()` (`main.cpp` line ~800)
3. **ACK/DENY** — Add handling in the ACK/DENY section (lines ~920-970)

16.5 Exposing Data to TCP Client

In the `loop()` TCP section, format data using `buildStatus()` or `buildDisplayPacket()` helpers. New telemetry fields should be added to `buildDisplayPacket()` for the Flask server.

17. INA226 Library (Legacy)

File: `lib/INA226/INA226.h` (109 lines)

A standalone I2C driver for the INA226 current/voltage sensor. Provides:

- `begin()` — Init with AVG=16, 1.1ms conversion, continuous shunt+bus
- `readBusVoltage()` — 1.25mV/bit resolution
- `readShuntVoltage()` — 2.5μV/bit resolution
- `readCurrent()` — Calculated from calibration register
- `readPower()` — Direct register read

Status: Likely **unused** in current firmware. The INA226 telemetry (INA status, charger current, voltages) now comes from the STM32 via the `STATUS2` UART message. This library appears to be a holdover from an earlier design where the ESP32 read the INA226 directly over I2C.

18. Known Constraints & Caveats

18.1 LVGL Float Formatting

`LV_SPRINTF_USE_FLOAT = 0` in the LVGL stdlib config. Any float formatting must use standard C `snprintf()`, not LVGL's internal `snprintf`. This is documented in the `ui.cpp` header comment.

18.2 Single TCP Client

Only one TCP client can be connected at a time. A new connection drops the existing one (line 2178-2184). The TCP idle timeout is 30 seconds.

18.3 Touch Coordinate Scaling

The GT911 reports raw coordinates in approximately 470×265 space. The touch filter scales these to the 800×480 display. This scaling is specific to the ESP32-8048S043C board and would need adjustment for different display modules.

18.4 NVS Dual-Location Lead Resistance

Lead resistance is stored in both `welddcfg/lead_r_mohm` and `spotwelder/lead_r` for backward compatibility. Any migration should update both locations or consolidate to one.

18.5 Boot Grace Period

A 2-second grace period after `sendBootConfig()` suppresses STATUS message recipe-sync. This prevents the STM32's initial default STATUS response from overwriting the ESP32's NVS-loaded recipe values. Be aware of this when debugging boot-time setting discrepancies.

18.6 LVGL Memory

128 KB LVGL heap at 800×480 resolution. Adding significantly more widgets or enabling additional fonts may require increasing `LV_MEM_SIZE`.

18.7 No Persistent WiFi Credentials

WiFi SSID and password are hardcoded at compile time. There is no runtime WiFi configuration UI.

18.8 Weld Recipe Editing While Armed

The Pulse tab lock was intentionally removed (`ui.cpp` line 1973 comment). Users can edit recipe drafts while armed, but must still click Apply to send changes. This is a design decision, not a bug.